

進捗報告

無線容量に追従する動的経路制御の実装と検証

60GHz 無線リンクの容量変動を実時間で推定し、WCMP の重みと HTB の整形レートを自動更新する制御ループを実装・検証した。
あわせて ECMP と容量比 WCMP を同一条件で比較した。

対象期間	2026-09-12 ~ 2026-09-18
前回報告	docs/RADWIN無線導入記録_20260911.pdf (2026-09-04 ~ 09-11)
実施した計測	10 条件 (うち有効 8 条件)
新規スクリプト	radwin_telemetry.py / radwin_ssh.py / radwin_wcmp_controller.py ほか
成果物	results/frr/radwin/dynamic/20260918_success_dynamic_control/
訂正 (2026-09-25)	§ 4.1・§ 4.3 の係数 0.52 の根拠を訂正 (無線の実効率は 0.568、2.4 Gbps は ODU の 2.5GbE ポートで頭打ちした値)

本書の数値はすべて results/frr/

配下の計測ファイルから集計した実測値である。推定値・モデル値にはその旨を明記する。

1. 前回からの到達点

前回報告の時点では、無線 2 ホップを CR1 経路に組み込み、単一経路で 3 シナリオを計測できた段階だった。本期間では 3 経路を同時に使う構成に進み、さらに無線容量の変動に自動追従する制御ループを実装して実測で検証した。

項目	前回 (09-11)	今回 (09-18)
経路の使い方	CR1 単独 (主経路 + 障害時フォールバック)	3 経路同時 (ECMP / 容量比 WCMP)
フロー数	4 本 / クラス (計 12 本)	64 本 / クラス (計 192 本)
無線容量の扱い	固定値 (CR1_BW を手で設定)	ODU テレメトリから実時間推定・自動適用
総スループット	0.899 Gbps (無線が律速)	20.14 Gbps (WCMP, 無線 2.16G 時)
障害時の AF41	21 秒の完全断	低下 1.6 %・断ゼロ (WCMP)

本期間で得られた主要な知見

- ① 経路の重みが容量を反映していないと、リンクごとの優先度制御 (HTB) が成立していてもエンドツーエンドの優先度差が消える。ECMP では AF42:AF43 = 0.98:1 となり、設計値 2:1 が失われた。
- ② 容量比 WCMP はこれを回復し (2.00:1)、同時に 全リンクを 100 % 利用した。
- ③ 無線容量は静止環境でも実際に変動する。実測で 493~2420 Mbps (4.9 倍) の幅があり、固定値による設定は原理的に追従できない。

なお本期間の作業を通じて、既存スクリプトに計測データの妥当性に関わる不具合を 2 件発見し修正した。詳細は第 5 章に記す。

2. 実験条件の整備 — 交絡要因の除去

3 経路を同時に使う比較を行うにあたり、結果を歪める要因を順に取り除いた。いずれも「効果があるはずの機構が効いていない」または「別の要因が結果を作っていた」ものである。

2.1 フロー数を 64 本 / クラスに増やした

マルチパスの分配はフロー単位のハッシュで決まるため、フロー数が少ないと分配粒度が粗くなり、重みどおりの配分にならない。従来の 4 本 / クラスでは 3 経路に分けきれないため 64 本に増やした (総送信量 24 Gbps は不変、1 本あたり 125 Mbps)。

128 本も試したが、iperf3 の接続確立が同時に集中して一部ストリームが失敗し (AF42 が 126/128 本)、計測が欠測した。64 本を採用し、ストリーム数の自動検査と再試行を `run_all_scenarios.sh` に組み込んだ。

2.2 入口ポリサーを外した

LER_Ingress の入口に総容量 × 4:2:1 のポリサーが入っており、経路に載せる前の段階でクラス間の差を作っていた。単一経路の計測ではリンク容量の方がはるかに厳しいため表面化しなかったが、3 経路構成では総容量が増えるためポリサーが支配的になる。

これを残したままだと「HTB による優先制御の効果」と「入口ポリサーの効果」を分離できない。INGRESS_POLICE=auto を導入し、マルチパス時は自動的に無効化するようにした。

2.3 経路設定が 3 箇所に分散していた

経路の設定が `frr_dscp_te.sh`・`frr_measure.sh`・`frr_te_monitor.sh` の 3 箇所にあり、後から実行される `frr_measure.sh` がシナリオごとに CR1 単独の経路で上書きしていた。このため ROUTE_MODE を `wcmp` にしても計測時には単一経路に戻っていた。3

箇所すべてを ROUTE_MODE に従うよう統一した。

2.4 L3+L4 ハッシュの有効化

Linux の既定 (fib_multipath_hash_policy=0) は送信元・宛先 IP

のみでハッシュするため、同一クラスの 64 本はすべて同じ経路に載ってしまう。ポート番号を含む L3+L4 ハッシュ (値 1) を明示的に有効化した。

この設定がないとマルチパスは一切機能しない

エラーは出ず、経路表には 3 本の nexthop が並んだままなので、設定ミスに気づきにくい。実際に分配されているかは path_stats.csv の経路別 TX バイト数で確認する必要がある。

3. ECMP と容量比 WCMP の比較

3 経路の容量は CR1 (無線) だけが大きく小さい。この 容量の非対称性 に対して、等分配 (ECMP) と容量比分配 (WCMP) がそれぞれどう振る舞うかを同一条件で比較した。無線容量が 0.9 Gbps の条件と 2.16 Gbps の条件の 2 水準で実施している (前者はアンテナ調整前、後者は調整後の実測値)。

条件	CR1 (無線)	CR2 / CR3 (有線)	重み CR1:CR2:CR3
ECMP	0.9 または 2.16 Gbps	各 9 Gbps	1 : 1 : 1
WCMP	同上	各 9 Gbps	容量比 (例 6:25:25)

送信は AF41 / AF42 / AF43 の 3 クラス × 64 フロー、各クラス 8 Gbps、合計 24 Gbps でリンク容量を意図的に超過させている (輻輳は本実験の前提条件)。

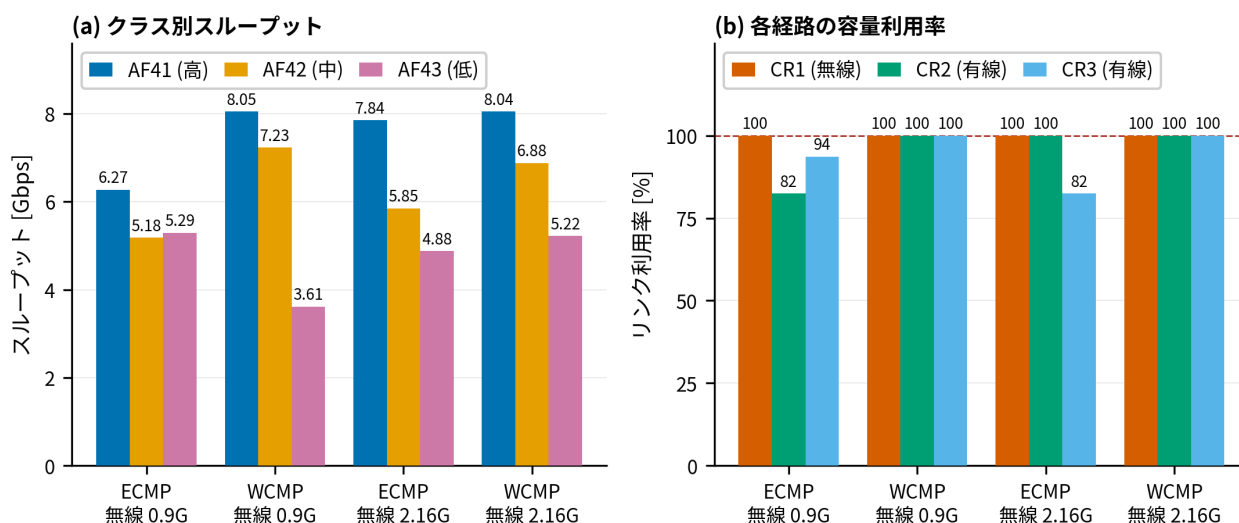


図 1 ECMP と容量比 WCMP の比較 (normal シナリオ、t = 5~55 s の平均)。(a) クラス別スループット。(b) 各経路の容量利用率。

3.1 ECMP は優先度制御を壊す

最も重要な結果は AF42 と AF43 の比である。HTB の quantum は 2:1 に設定してあり、輻輳時には AF42 が AF43 の 2 倍のスループットを得るのが設計値である。

条件	AF41	AF42	AF43	AF42 : AF43	合計
ECMP (無線 0.9G)	6.268	5.181	5.294	0.98 : 1	16.744
WCMP (無線 0.9G)	8.048	7.228	3.614	2.00 : 1	18.890
ECMP (無線 2.16G)	7.845	5.846	4.876	1.20 : 1	18.567
WCMP (無線 2.16G)	8.045	6.875	5.223	1.32 : 1	20.143

単位はすべて Gbps。

ECMP (無線 0.9G) では AF42 : AF43 が 0.98 : 1 となり、優先度の差が完全に消えている。各リンク上では HTB が正しく 2:1 で分配しているにもかかわらず、である。

原因は、ECMP が容量を見ずに 1/3 ずつ送るため、0.9 Gbps の無線経路に 1/3 のトラフィックが押し込まれて大量に廃棄されることにある。この廃棄はクラスに関係なく起きるため、リンク単位で作った優先度差が経路選択の段階で打ち消される。

経路選択が先、スケジューリングが後

各リンクの HTB がどれだけ正しく動いていても、そこへ到達するトラフィックの割り当てが容量を無視していれば、エンドツーエンドの優先度は保証されない。優先度制御は経路選択と独立には設計できない。

なお無線容量が 2.16 Gbps の条件では ECMP でも 1.20:1 まで回復している。経路間の容量格差が 10 倍から 4.2

倍に縮まったため、容量を見ない分配の害は経路間の容量格差に比例して大きくなることを示している。

3.2 容量利用率

図 1(b) のとおり、WCMP は 2 条件とも 3 経路すべてを 100 % 利用した。一方 ECMP は無線経路を 100 % 使い切る一方で有線側が余っており（無線 0.9G 条件で CR2 = 82 %、無線 2.16G 条件で CR3 = 82 %）、総スループットで 12.8 % / 8.5 % の差になった。

ECMP で有線が余るのは、無線に割り当てられて廃棄されたトラフィックが他経路へ回されないためである。等分配は「公平」に見えて、容量が非対称な場合には全体の利用効率も下げる。

3.3 障害時の耐性

t = 20 s に CR1 (無線) へ 100 % 損失を注入し、t = 40 s に解除する。下表は障害中 (t = 22~39 s) の AF41 スループットを障害前 (t = 5~19 s) と比較したものである。

構成	障害前	障害中	変化	AF41 の断
単一経路 (CR1 のみ)	0.861	0.000	-100 %	21 秒
ECMP (無線 0.9G)	5.011	4.148	-17.2 %	0 秒
WCMP (無線 0.9G)	8.048	7.918	-1.6 %	0 秒
ECMP (無線 2.16G)	8.037	6.158	-23.4 %	0 秒
WCMP (無線 2.16G)	8.045	6.786	-15.6 %	0 秒

単位 Gbps。迂回なし (failure シナリオ) の値。OSPF-SR による迂回を有効にした failure_reroute では、WCMP・ECMP とも AF41 の低下は 1 % 未満に収まる。

単一経路では 21 秒間の完全断が生じるのに対し、3 経路構成ではいずれも断がない。無線容量が 0.9 Gbps の条件で WCMP の低下が 1.6 % にとどまるのは、WCMP がもともと無線経路に全体の 4.8 % しか載せていないためである。無線容量が 2.16 Gbps では無線の担当分が 10.7 % に増えるため、失う量も増えて -15.6 % となる。

重み付けは障害耐性も同時に与える

容量比 WCMP は、脆弱なリンクに対して自動的に「そのリンクが落ちても影響が小さい」配分を与える。冗長化のための追加機構ではなく、容量に応じた配分という一つの原理から帯域効率と障害耐性の両方が得られる。

3.4 再現性

同一条件を 3 回 (normal / failure / failure_reroute の障害前区間) 計測したときの AF41 のばらつきを比較すると、両者の差は大きい。

条件	3 回の AF41	幅
ECMP (無線 0.9G)	6.268 / 5.011 / 6.645 Gbps	± 14 %
WCMP (無線 0.9G)	8.048 / 8.048 / 8.048 Gbps	± 0.02 % 未満

ECMP のばらつきは、どのフローがハッシュで無線経路に落ちるかが試行ごとに変わるためである。容量の小さい経路に載ったフローは大量に廃棄されるので、どのクラスのフローが何本そこに当たったかが結果を左右する。WCMP では無線経路の担当分が容量に見合っているため、この感度が消える。

実験の再現性という観点からも、容量を反映した重み付けは必要である。

4. 無線容量に追従する動的制御

第 3 章の WCMP は重みを手で与えている。しかし無線容量は固定ではない。重みと HTB の整形レートを実際の無線容量から自動生成する 制御ループを実装した。

4.1 ODU テレメトリの取得

V90 の各 ODU から `iw dev wlan0 station dump` を実行し、PHY レート・MCS・受信電力を読む。ODU の SSH は `dropbear v2019.78` で、公開鍵認証が (正しく配置しても) 拒否されたため、Python の `pty.fork()` によるパスワード自動応答で接続している。パスワードは `~/radwin_pass (600)` に置き、スクリプトには埋め込まない。

チェーン全体の実効容量は次式で推定する。2 ホップは直列なので小さい方が律速し、TDD 半二重などのため実効値は PHY レートより大きく下がる。

[2026-09-25 訂正] 当初は係数 0.52 を「PHY の約 52 % が実効値」と説明したが、これは無線の効率ではない。無線そのものの効率は、MCS 12 (PHY 4620 Mbps) で ODU 内蔵 SpeedTest 2624 Mbps、`prs-link-estimator` の `currentThroughput` 12 回平均 2623 Mbps から 0.568 と実測した。0.52 は、低い MCS での効率を測っていないため、0.568 より保守側の値として残している。

```
C_chain = min(PHY_hop1, PHY_hop2) × 0.52
CR1_BW = C_chain × 0.9                (安全率)
w1      = round(CR1_BW / 9000 × 100) (w2 = w3 = 100 に固定)
```

4.2 制御則 — 非対称なヒステリシス

容量の推定値が現在値から ±10 % 以上変化したときに更新する。ただし 下げと上げで扱いを変える。

方向	条件	理由
下げ	1 回の観測で即座に適用	容量を過大評価したままだと無線が溢れて損失が出る。安全側に倒す
上げ	3 回連続で確認してから適用	一時的な改善で上げると、直後に戻ったとき過大評価になる

リンクが完全に切れた場合は制御ループでは経路を触らない。OSPF と `frr_te_monitor.sh` が担当しており、二重に操作すると競合するためである。

また、前提条件 (HTB `qdisc` の存在・経路が 3 本 `nexthop` であること・L3+L4 ハッシュが有効であること) を起動時に検査し、未達なら 適用せず中止する。適用が失敗した場合も内部状態を進めず、ログに `_FAIL`

を残して次回再試行する。

「制御したつもり」を残さない

tc や ip route は対象が無くても静かに失敗することがある。検査と失敗記録がないと、何も適用されないまま正常なログだけが残し、後から見て区別がつかない。実際にこの検査が、前回計測の残骸で HTB が消えていた状態を検出した (5.1 節)。

4.3 実測による検証

トラフィックを流した状態で制御ループを動作させ、途中で手動でアンテナの向きを変えて容量を変動させた。網掛けが手動操作を行った区間である。

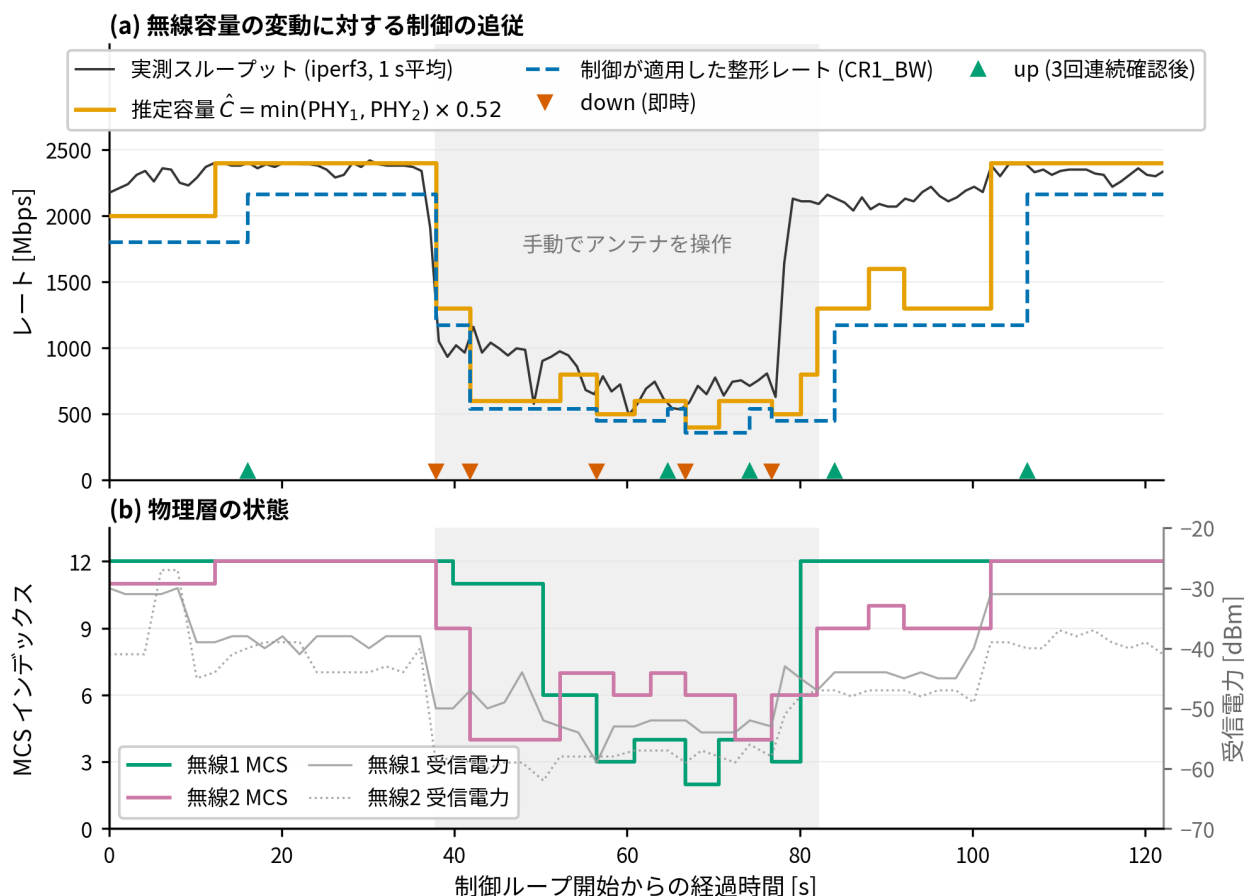


図 2 無線容量の変動に対する制御の追従。(a) 実測スループット・推定容量・適用した整形レート。(b) 各ホップの MCS と受信電力。網掛けは手動でアンテナを操作した区間。

項目	結果
観測時間 / サンプル数	122 秒 / 62 サンプル (2 秒間隔)
実測スループットの変動	493 ~ 2420 Mbps (4.9 倍)
制御動作	down 5 回 / up 5 回 / keep 37 回
適用失敗	0 回
低下の検知遅れ	1 ポーリング以内 (☒ 2 秒)
上昇の反映遅れ	3 ポーリング (5.7 秒) — 設計どおり
却下された上昇	7 回中 2 回 (確認中に容量が戻らず、正しく見送り)

特筆すべきは 7 回の上昇候補のうち 2 回が却下されたことである。いずれも 2 回目の確認までは条件を満たしたが、3 回目までに容量が戻らなかった。即座に上げていれば過大評価となっ

ていたケースであり、非対称ヒステリシスが実データで機能することを示している。フラッピングは一度も発生していない。

また、容量を最大値（2162M）まで上げた状態が最も安定していた（MCS 12/12 を 20 秒維持）。「容量を上げる → トラフィック増 → 干渉 → 容量低下」という正のフィードバックは観測されなかった。

4.4 推定式の精度と限界

推定容量と実測スループットの相関係数は $r = 0.891$ であった。ただし精度は状況によって大きく異なる。

区間	推定	実測	誤差
安定時（MCS 12/12、24 サンプル）	2402 Mbps	2360 Mbps	-1.7 %
回復直後（ $t = 84 \sim 100$ s）	1301 Mbps	2130 Mbps	-39 %（過小評価）

安定時は推定と実測の差が 1.7 % であった。[2026-09-25 訂正]

当初はこれをモデルの独立した裏付けとしたが、誤りである。実測 2360 Mbps は ODU の 2.5GbE ポート（実効約 2475 Mbps）で頭打ちした値で、無線はそれより多く運べる（上記の 2624 Mbps）。一致は偶然であり、安定時の推定（2402 Mbps）は無線の実容量に対し約 8 % 保守側である。

一方、回復の過渡状態では大きく過小評価する。図 2(a) で実測（黒線）は $t \approx 79$ s に 2100 Mbps へ跳ね上がっているが、適用値（青破線）が追いつくのは $t = 106$ s で、27 秒の遅れがある。この間、無線は平均 2170 Mbps 出せたのに制御は 1171M しか許可していない。

原因は図 2(b) に現れている。iw が返す tx bitrate

は直前に送信したフレームのレートであり、レート適応が段階的に MCS

を上げていく過程をそのまま反映する。実際に出せるスループットより低い値が続くため、推定が遅れる。

安全側の誤りだが、無線容量を捨てている

過小評価なので通信が途切れることはない（余剰は有線経路が吸収する）。しかし 27 秒間にわたり無線容量の約半分を使わずにいた。改善案は、下げは MCS 基準のまま（速く安全側）、上げは実測スループット基準に変えること。経路別の送信バイト数は既に path_stats.csv で取得しているため、同じ手段で実装できる。

5. 発見した不具合と修正

本期間の作業を通じて、既存スクリプトに計測データの妥当性に関わる不具合を発見した。いずれもエラーを出さずに進行するため、結果を見ただけでは気づけない種類のものである。

5.1 障害注入が HTB を破壊していた（データ妥当性に影響）

frr_measure.sh の障害注入は `tc qdisc add dev leri-cr1 root netem loss 100%`

で行っており、これが HTB の root qdisc を置き換えていた。さらに $t = 40$ s の解除処理が `tc qdisc del ... root` を無条件に実行するため、netem もろとも HTB が消え、再設定されないまま計測が続いていた。

検出は経路別送信レートの比較による。障害前は CR1

が設定値ちょうどに張り付くのに対し、復旧後は必ず設定値を超えていた。

計測	CR1_BW 設定	障害前（ $t = 15 \sim 19$ s）	復旧後（ $t = 45 \sim 55$ s）
20260918_wcmp / failure_reroute	0.900 G	0.900 Gbps	2.013 Gbps
20260918_wcmp / failure	0.900 G	0.900 Gbps	1.132 Gbps
20260918_wcmp_ref / failure_reroute	2.160 G	2.160 Gbps	2.444 Gbps
20260918_wcmp_ref / failure	2.160 G	2.160 Gbps	3.272 Gbps

障害前が小数第 3 位まで設定値と一致し、復旧後だけが超過している。整形が失われた証拠である。

修正: netem を HTB の root ではなく 葉クラス (1:1 / 1:2 / 1:3) に付ける方式へ変更した。解除しても HTB が残る。1:3 は HTB の default クラスでもあるため、fwmark の付かない OSPF hello もここに落ち、隣接断の再現性は保たれる。

過去データへの影響範囲

影響なし: normal シナリオ全体 (netem を使わない)。failure 系の $t = 0 \sim 40$ s (障害検知・迂回の評価は有効)。
無効: failure 系の $t \geq 43$ s (復旧後)。無線経路が無整形になり CR1 が設定容量を超える。本報告の第 3.3 節は $t = 0 \sim 40$ s のみを集計している。復旧後の数値を用いる場合は再計測が必要である。

5.2 HTB の default クラスが存在しない

frr_dscpt_te.sh は htb default 13 を指定しているが、実在するクラスは 1:0 / 1:1 / 1:2 / 1:3 のみで 1:13 は存在しない。HTB は default が存在しないクラスを指すと未分類パケットを direct queue へ流すため、整形を一切受けない。

本番計測では全トラフィックが fwmark または MPLS (0x8847) のフィルタにマッチするため実害はない。ただし アドホックな帯域プローブは素通りする。実際、CR1_BW を 360~540M に設定していた約 42 秒間も、fwmark のない TCP は一度も 0.6 Gbps を下回らず、最大 2.420 Gbps まで出た。

HTB の効きを検証する際は必ず fwmark 付きのトラフィック、または MPLS 経由の本番パスを使うこと。

5.3 無線の MCS はトラフィックがないと上がらない

不具合ではないが、計測手順に影響する重要な性質である。802.11ad のレート適応は実データの成功率を観測して MCS を決めるため、通信がないと低い MCS に留まったまま動かない。

実測では、無通信の状態で 72 秒間 MCS 9 のまま変化しなかったものが、iperf3 を流した 12 秒後には MCS 12 に達した。

切り分けは受信電力と MCS の不一致で行う

この現象は「アンテナを元に戻したのにスループットが戻らない」という形で現れ、アライメント不良と誤診しやすい。決め手は受信電力である。

実測例: 無線 1 = MCS 12 / -30 dBm、無線 2 = MCS 9 / -28 dBm。信号が良い方の MCS が低い伝搬の問題ではないと確定できた。この判断のため、制御ループの記録項目に受信電力を追加した。

6. 現時点の到達点と残課題

最終目標は「障害や輻輳が発生しても、自動で検知してリルーティングと輻輳制御を行い、途切れない通信を実現する」ことである。現時点の達成状況は次のとおり。

要素	状況	根拠
輻輳下の優先度制御	達成	WCMP で AF42:AF43 = 2,00:1 (設計値どおり)
リンク障害の自動迂回	達成	failure_reroute で AF41 低下 1 % 未満・断ゼロ
容量低下の自動検知	達成	MCS 変化を 2 秒以内に検知、誤検知ゼロ
容量に応じた自動再配分	達成	重みと HTB を自動更新、適用失敗ゼロ
本番トラフィックでの統合検証	未実施	制御ループと 3 クラス計測は未だ別々に実行
回復時の追従速度	要改善	27 秒の遅れ (4.4 節)

6.1 次に行うべきこと

- (1) 本番トラフィックを流した状態での統合検証。現時点では制御ループの検証（単一フローのプロープ）と 3 クラス計測が別々に行われている。AF41 / AF42 / AF43 を流しながら無線容量を変動させ、容量が変わっても優先度制御が維持され、AF41 が途切れないことを示す必要がある。これが最終目標の直接的な証拠になる。
- (2) 回復時の追従改善。 上げ方向の判断を実測スループット基準に変更する（4.4 節）。ただし（1）を先に行えば、現在の 27 秒の遅れが優先度制御にどの程度影響するかを実測できる。改善後との比較対象になるため、（1）を先に実施することを推奨する。
- (3) failure 系シナリオの再計測。 5.1 節の修正後、復旧後区間を含む完全なデータを取り直す。
- (4) 20260918_ecmp_ref の再計測。 AF43 が 62/64 本しか接続できず欠測している（normal シナリオが無効）。

6.2 成果物の所在

results/frr/radwin/dynamic/20260918_success_dynamic_control/	本報告の図 1・図 2 と生成スクリプト
results/frr/radwin/comparison/20260918_wcmp/, _wcmp_ref/	WCMP 計測（無線 0.9G / 2.16G）
results/frr/radwin/comparison/20260918_ecmp_nopolice/, _ecmp_ref2/	ECMP 計測（同上）
scripts/radwin_telemetry.py	ODU テレメトリ取得と容量推定
scripts/radwin_wcmp_controller.py	制御ループ本体
scripts/radwin_ssh.py	dropbear 向け pty パスワード SSH
results/frr/tools/plot_dynamic_control.py, plot_ecmp_wcmp.py	図の生成
/tmp/radwin_controller.csv	制御ループのログ（図 2 の元データ）

本報告の数値はすべて上記ディレクトリの計測ファイルから集計した。集計に用いた区間・条件は各節に明記している。